

Q2 — CAN Bus & FlexRay

MM2 · Hans-Peter Schwefel

We look at two fieldbus protocols used in embedded/automotive systems:

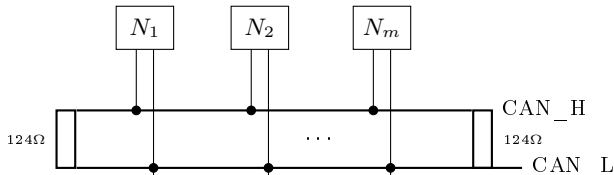
1 Fieldbus Overview

Fieldbus = single closed network operating mainly at **OSI layers 1&2** (sometimes also layer 7). nodes share one bus.

2 CAN Bus — Physical Layer & Arbitration

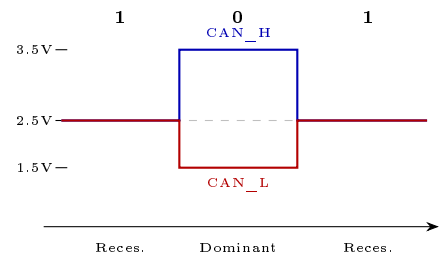
All nodes connect to a shared **differential pair** bus (CAN_H and CAN_L), terminated at each end with a 124Ω resistor to prevent signal reflections. Bits are encoded as voltage differences between the two lines.

Bus topology (differential pair, shared):



Why differential pair? Noise hits both wires equally — receiver reads the *difference*, so common-mode interference cancels out. Crucial in noisy automotive environments.

Voltage levels (recessive = 2.5 V both, dominant = spread):



Recessive (1): both lines at 2.5 V
Dominant (0): CAN_H $\uparrow$ 3.5 V, CAN_L $\downarrow$ 1.5 V
Dominant **always overwrites** recessive.

2b CAN ID & Arbitration

Since dominant (0) always overwrites recessive (1), priority is determined by the CAN ID: the node with the **lowest ID wins** the bus automatically.

Arbitration — lowest ID wins:

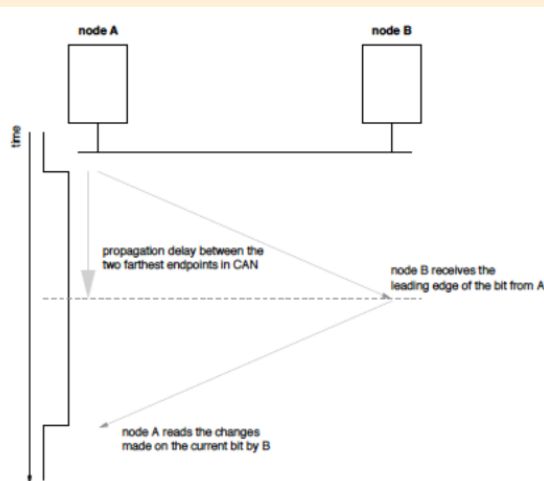
- Only Tx when **bus is idle**
- All nodes send CAN ID simultaneously
- Read back bus while sending; if read $\neq$ sent $\Rightarrow$ **stop**
- Dominant (0) overwrites recessive (1)
- Messages have IDs $\Rightarrow$ diff. priority msgs
- **Error detection:** error $\Rightarrow$ retransmit

Example — 3 nodes transmit simultaneously:

	b1	b2	b3	b4	Result
N ₁	0	0	0	0	Sends
N ₂	0	0	1		Stops (b3)
N ₃	0	0	0	1	Stops (b4)

Relates to Q4 (Network Calculus): CAN/FlexRay traffic can be modelled as arrival curves — used to compute worst-case delay bounds for safety-critical messages.

Note CAN Bit Stuffing, Bit Timing & RTT (Don't say only for remember)



RTT constraint: A $\rightarrow$ B $\rightarrow$ A must fit within 1 bit time so arbitration works. Limits max bus length.

1 Mbit/s $\Rightarrow$ 1 μ s $\Rightarrow$ $\approx$ 40 m
125 kbit/s $\Rightarrow$ $\approx$ 500 m

Bit stuffing: after 5 identical bits, CAN inserts an opposite *stuff bit*.

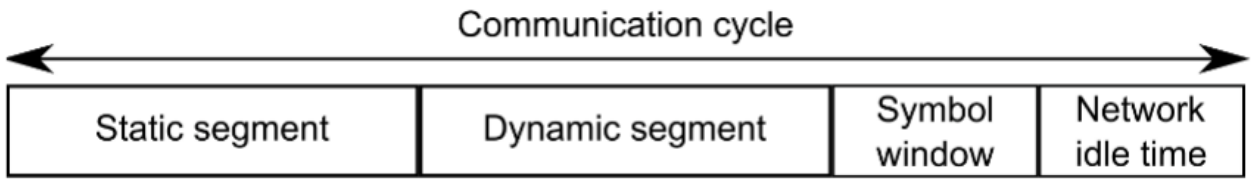
Why: guarantees transitions for receiver clock recovery.

Bit timing: each bit = time quanta (TQ). Sample point programmable within the bit — must account for propagation delay (RTT $\leq$ 1 bit time).

3 FlexRay — Communication Cycle

Unlike CAN’s event-driven arbitration, FlexRay is **time-triggered**: every node follows a fixed repeating **communication cycle** split into 4 segments.

- THE CYCLE LENGTH IS DETERMINED BY THE **SHORTEST PERIODIC REQUIREMENT** IN THE SYSTEM — ALL TIMING IS PRE-PLANNED, GIVING DETERMINISTIC WORST-CASE GUARANTEES.



Static Segment (always starts at action point)

- Divided into **equal-size slots**, node always transmits in its slot
- # slots per node depends on node bandwidth use
- Worst-case delay: **at most 1 cycle**

Symbol Window

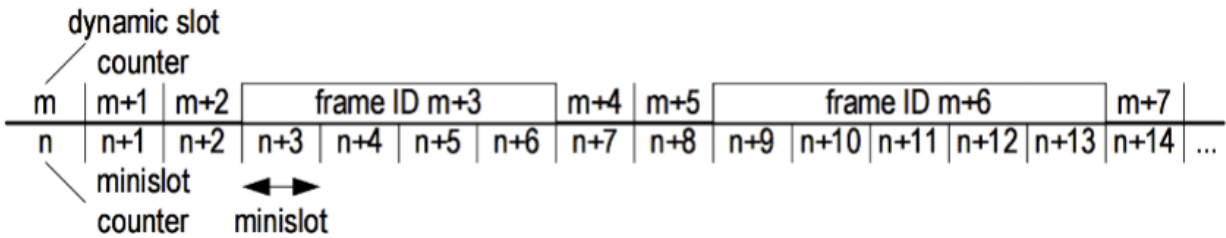
Special bit pattern transmitted, e.g. **waking up a node**.

Dynamic Segment (priority on node ID)

- Always same # mini-slots but **different dynamic slot counter**
- Always fills **1 mini-slot** even when not transmitting
- Each node has **pLatestTx** (latest possible Tx time) — if not reached in time $\Rightarrow$ wait next cycle

Network Idle Time

Corrects **microtick** adjustments to align with **macrotick** perfectly (clock sync).



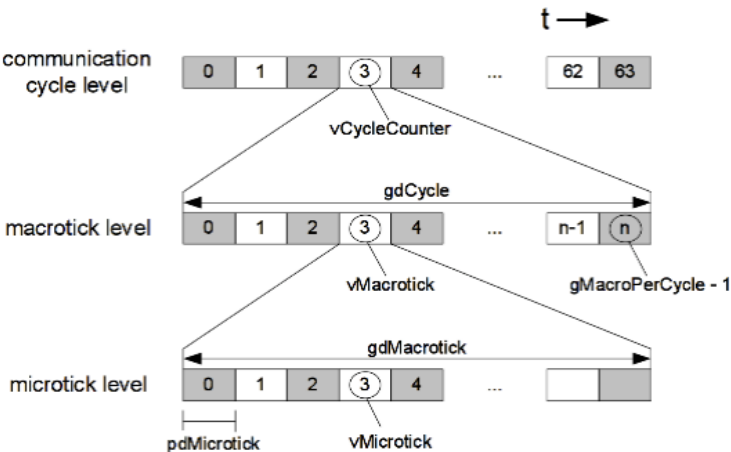
Rest Extra Here Under

4 CAN vs FlexRay — Use Case Comparison

	CAN	FlexRay
Cost	Cheap	Expensive
Throughput	Lower rate	Higher rate
Traffic	Bursty / event-driven	Periodic + bursty
Timing	Not life-critical	Exact, deterministic
Reliability	—	2-channel redundancy
Use case	Body electronics, sensors	Chassis, safety-critical (X-by-wire)

Key distinction: CAN uses *arbitration* (event-driven, non-deterministic worst-case); FlexRay uses *TDMA* (time-triggered, guaranteed slots, deterministic worst-case delay).

6 FlexRay — Timing Hierarchy & Clock Sync



Microtick = smallest hardware clock unit (node-local oscillator).
Macrotick = agreed global time unit, built from microticks.
NIT at end of each cycle corrects drift: nodes adjust how many microticks per macrotick so all stay aligned.

CAN has **no global clock** — sync per-bit via edge detection. FlexRay NIT maintains a **fault-tolerant distributed clock** $\Rightarrow$ deterministic slot timing.